

Load Test Execution Report

PG Webhook Service · Data Ingestion Workflow · 150,000 VU Profile (Compressed Preview Run)

Spec: pg-webhook.volume150k.js

Run ID: VOL150K-QUICK-TEST

Date: 2026-07-06

Introduction

What is a load test?

A load test checks how a system behaves when many people or systems use it **at the same time**. Instead of waiting for real-world traffic spikes to reveal problems, we simulate that traffic on purpose, in a controlled way, so issues can be found and fixed before customers ever notice them.

What was being tested?

This test targeted the **PG Webhook Service** inside RMS (Revenue Management System) Kenya, the component responsible for receiving payment-gateway (PG) notifications and turning them into recorded transactions. Each simulated user follows a two-step workflow: first it calls the **Sign API** (/pgwebhook/dev/sign) to obtain a valid signature, then it calls the **Transaction API** (/pgwebhook/transactions) to submit the signed transaction for ingestion. The goal was to see whether this workflow could keep up when **150,000 users** tried to do this at once.

How was it tested?

The test was scripted in **k6** (an industry-standard load-testing tool) using a spec called `pg-webhook.volume150k.js`. The script gradually “ramps up” simulated users, called Virtual Users or **VUs**, from 0 up to a peak of 150,000, holds them there, then ramps back down to 0, while measuring response times and success and failure rates at every step. This particular run was a **quick, compressed preview** of that full profile (30 seconds nominal instead of the full ~60-minute production run) executed from a **single ordinary workstation**, purely to get an early read before committing to the full-length test.

What was the result?

The workstation used to generate traffic could only sustain **20,572 concurrent users**, about 14% of the 150,000 target, before it ran out of its own connection capacity. This caused a high number of failed and slow requests, but the evidence points to the **test machine's limits, not RMS's**: every transaction that successfully obtained a signature was processed correctly (100% success), and response times for both endpoints stayed within acceptable thresholds. In short, **this run tells us the load generator needs to be upgraded before we can find out how RMS itself performs at 150,000 users**. The details and recommended next steps follow below.



CLIENT CAPACITY EXCEEDED: NOT A VALID BACKEND MEASUREMENT

PEAK VUs ACHIEVED

20,572

of 150,000 targeted (13.7%)

INGESTION
THROUGHPUT**45.68 /s**

11,276 successful ingestions

HTTP FAILURE RATE

56.58%

threshold: < 5%

ACTUAL DURATION

4m 07s

nominal target: 30s

**ENVIRONMENT LIMITATION: READ BEFORE ACTING ON THESE NUMBERS**

This run executed from a **single, unmodified Windows workstation**, not a distributed or otherwise properly resourced load-generation environment. A single machine's outbound connections to one destination are hard-capped by the TCP ephemeral port space, roughly 28,000 to 64,000 ports depending on OS tuning, far short of the 150,000 concurrent connections this profile targets. Concurrency plateaued at **20,572 VUs** and never grew further; the reported **56.58%** HTTP failure rate and **54.30%** application error rate are very likely dominated by the load generator running out of sockets, not by the RMS backend rejecting traffic. The 8× wall-clock overrun (4m 07s actual vs. 30s nominal) is the client's graceful-shutdown queue draining tens of thousands of stuck connections, one 30-second timeout at a time. **Treat this run as a characterization of this workstation's ceiling, not of the application's.** A credible reading at 150,000 VUs requires distributed k6 execution across multiple pods and IPs (see Next Steps).

Test Configuration

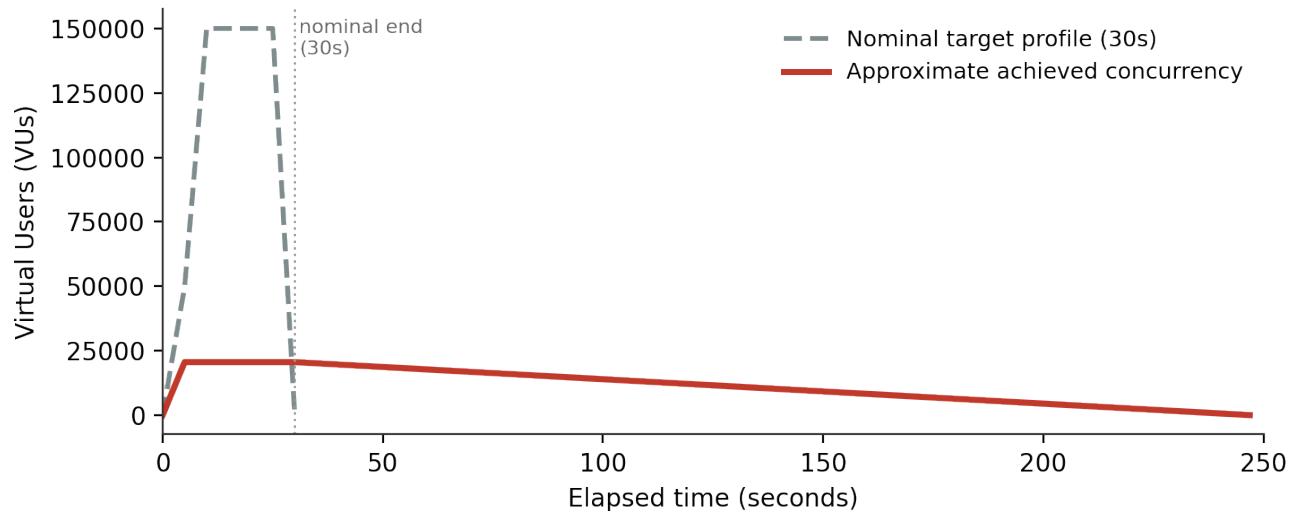
This was a deliberately compressed, low-duration preview of the volume150k spec, run to observe behavior at 150,000 target VUs without committing to the full ~60-minute production profile.

Peak Target Concurrency	150,000 VUs
Nominal Test Duration	30 seconds
Actual Wall-Clock Duration	246.9 seconds (4m 06.9s)
Load Generator	Single workstation (non-distributed)
Full Production Profile (reference)	0 → 50,000 → 150,000 VUs, held 50 min (~60 min total)

Load Profile (Compressed, Nominal)

0:00 to 0:05	Ramp: 0 → 50,000 VUs
0:05 to 0:10	Ramp: 50,000 → 150,000 VUs
0:10 to 0:25	Sustained peak (target): 150,000 VUs
0:25 to 0:30	Ramp down: 150,000 → 0 VUs

Load Profile: Nominal Target vs. Achieved Concurrency

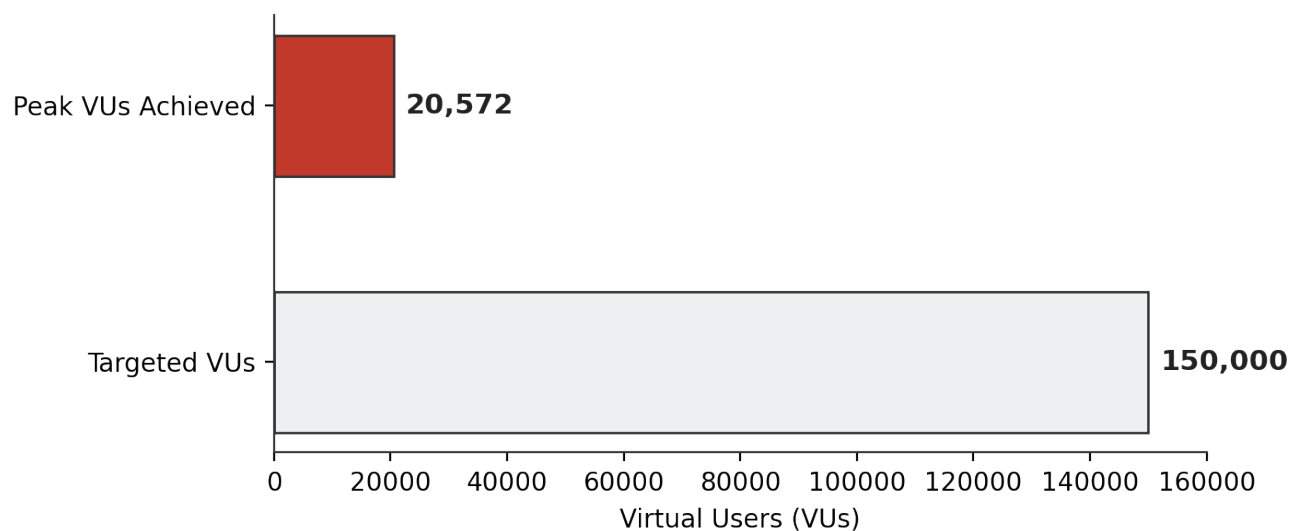


The dashed line shows the nominal target ramp (0 to 150,000 VUs over 30 seconds); the solid line shows the approximate concurrency the workstation actually sustained before its connection ceiling was reached.

Performance Summary

Metric	Count	Rate (over 246.9s actual)
Workflow Iterations Executed	40,465	163.9 /s
Total HTTP Requests	68,766	278.6 /s
Successful End-to-End Data Ingestions	11,276	45.68 /s

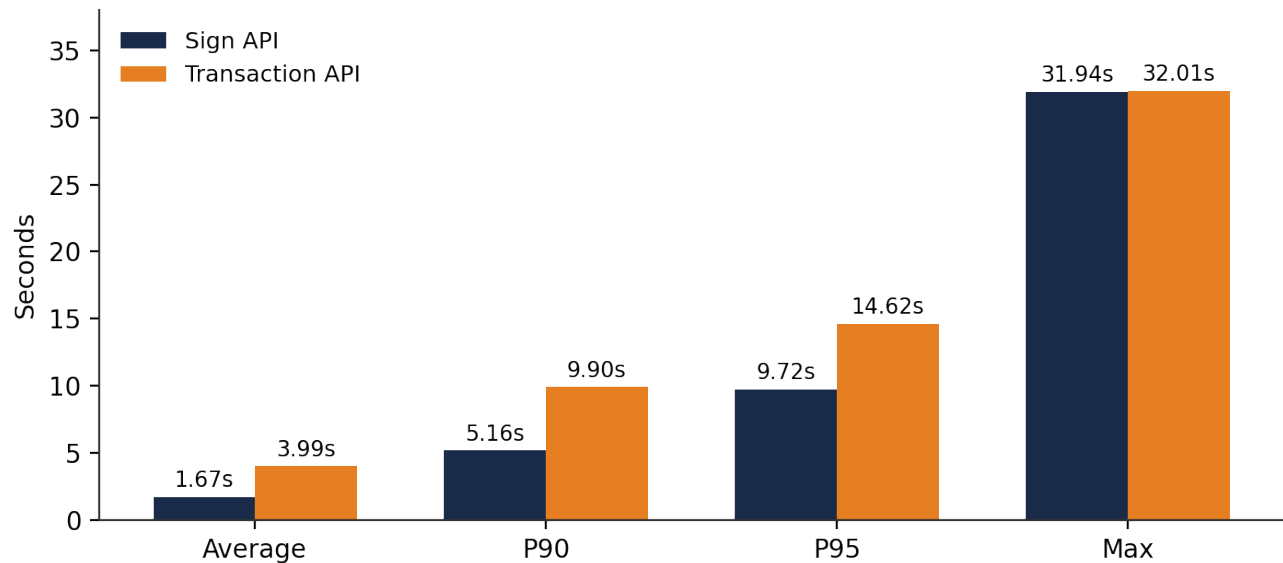
Concurrency: Targeted vs. Achieved



Response Times

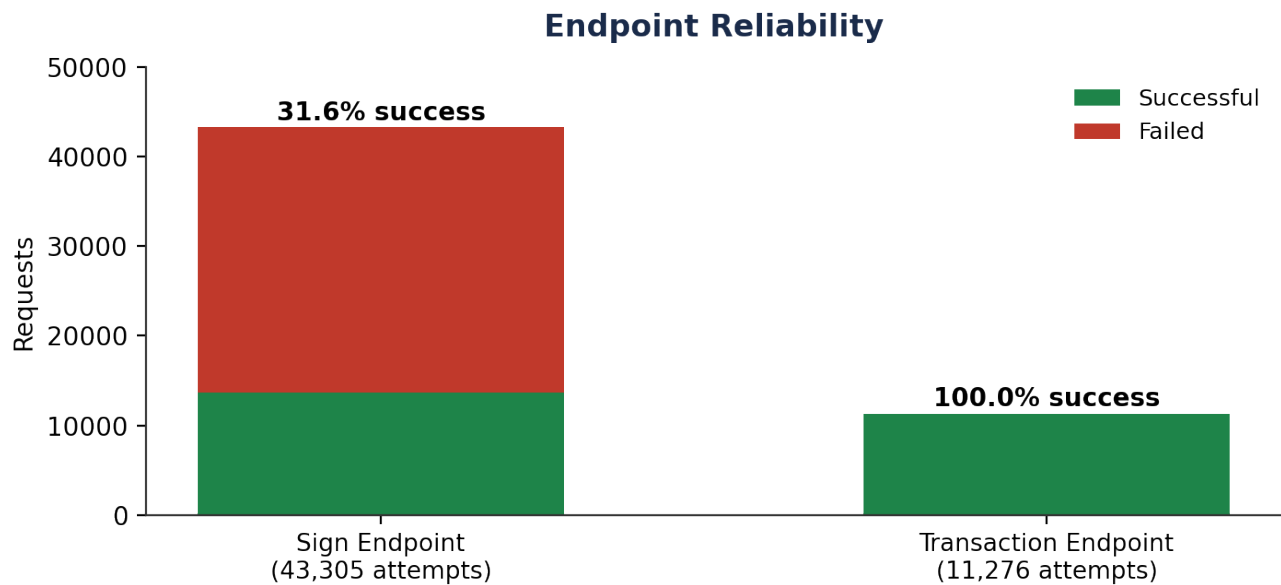
Endpoint	Average	P90	P95	Max
Sign API (/pgwebhook/dev/sign)	1.67s	5.16s	9.72s	31.94s
Transaction API (/pgwebhook/transactions)	3.99s	9.90s	14.62s	32.01s

Response Times by Endpoint



Reliability

Metric	Result	Threshold	Status
Sign Endpoint Success Rate	31.6%	N/A	13,695 / 43,305
Transaction Endpoint Success Rate	100.0%	N/A	✓ 11,276 / 11,276
HTTP Request Failure Rate	56.58%	< 5%	✗ FAILED
Application (PG) Error Rate	54.30%	< 5%	✗ FAILED
Sign Duration P95	9.72s	< 10s	✓ PASSED
Transaction Duration P95	14.62s	< 15s	✓ PASSED



Key Observations

- The load generator sustained a maximum of **20,572 concurrent VUs**, about 14% of the 150,000 target, before running out of client-side connection capacity.
- Of 43,305 sign-endpoint attempts, only **31.6%** succeeded; the sign step is the dominant point of failure, consistent with earlier, properly-scaled 5,000-VU tests against the same environment.
- Every request that received a valid signature was ingested successfully: the transaction endpoint held a **100% pass rate** with no data-integrity issues observed downstream of a successful sign.
- Latency thresholds passed for both endpoints (sign P95 9.72s, transaction P95 14.62s): the failures are a volume and connection problem, not a slowness problem.
- The nominal 30-second test required **4 minutes 7 seconds** of real wall-clock time, entirely due to the graceful-shutdown tail waiting on stuck connections.

Next Steps

- 1 **Re-run via distributed k6 execution.** Split the 150,000 VUs across multiple pods (each with its own IP) using k6's `--execution-segment`, so no single client hits the port-exhaustion ceiling before the backend does.
- 2 **Apply OS-level socket tuning** on every load-generation node: widen the ephemeral port range, enable `tcp_tw_reuse`, and raise the file-descriptor ulimit.
- 3 **Investigate the sign endpoint specifically.** It has now failed first in every scale of test run against this environment, from 5,000 VUs upward.
- 4 **Re-baseline once distributed execution is in place**, and treat this run's numbers strictly as a load-generator capacity finding, not an application capacity finding.